Unity Gameplay Developer Take-Home Test

Expected Time: 2 to 4 hours

Challenge Overview

Build a modular card drawing and playing loop using data-driven architecture. ***The game is complete nonsense***, but it works like so:

Players draw playing cards from a deck to form a hand, with a maximum hand size of 5 cards. They place cards from their hand onto the play area next to any previously placed card. At any time they can discard cards and replenish their hand with cards from the deck.
The game ends when all cards have been placed in the play area or discarded. Once this happens, the player receives a score based on the sum of all the cards shown on the play area.

Again: ***The game design here is nonsense. Don't stress about it.*** Just ensure that the few restrictions given are implemented accurately.

You will receive a starter Unity project containing:

- An asset pack containing a deck of standard playing cards
- A canvas layout with a Hand Panel, a Play Area, and a Discard Zone

Requirements

1. Data & Architecture

- Define a Card using ScriptableObjects to hold data (Suit, Value, etc.).
- Create a Deck Manager that holds a list of these ScriptableObjects and shuffles them at start.

2. Core Gameplay Loop

- Draw: Implement a UI button that draws a card from the deck and adds it to the player's Hand Panel.
- Hand Layout: Dynamically instantiate cards into the hand. Ensure cards lay out cleanly next to each other.
- Play: Allow the player to drag and drop a card from their hand onto the Play Area. If no previous cards have been played, reposition it to the center of the Play Area. All subsequent cards have to be placed in a blank space to the left, right, above, or below another card.
- Discard: Allow the player to drag and drop a card from their hand into the Discard Zone.

3. Game Conclusion

- Once all cards have been placed in the Play Area or the Discard Zone, the game ends.
- Total the value of all cards played on the Play Area.

4. State & Edge Cases

- Enforce a maximum hand size of 5 cards. Prevent drawing if the hand is full.
- Prevent users from placing cards on top of existing cards and anywhere not adjacent to an existing card.

5. Code Quality & Performance

- Update the hand UI using C# events instead of checking arrays inside Update().
- Write clean, self-documenting C# code with proper separation of UI and data logic.

Deliverables

- A link to a private Git repository (GitHub/GitLab).
- A short README.md file explaining your architecture choices.